A

Software Requirements Specification (SRS) and Design Document

ON

"AquaSense"

SUBMITTED BY

MR. Shreyash Chougule	24UCS023
MR. Krish Mahajan	24UCS056
MR. Pranav Chougule	24UCS021
MR. Varad Dake	24UCS024

UNDER THE GUIDANCE OF

V.V. KHERADKAR

D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji

(An Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Department of Computer Science and Engineering

2025-26

D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji

(An Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

Department of Computer Science and Engineering

CERTIFICATE

MR. Krish Mahajan	**24UCS056**
MR. Shreyash Chougule	**24UCS023**
MR. Pranav Chougule	**24UCS021**
MR. Varad Dake	**24UCS024**

Have successfully completed the SRS and Design work, of the mini project entitled,

AquaSense

In partial fulfillment for Second Year B.Tech. CSE. This is the record of their work carried out during academic year 2025-2026.

Date: **Place: Ichalkaranji**

V.V. KHERADKAR
(PROJECT GUIDE)

PROF. DR. D.V. KODAVADE
(HOD CSE DEPT.)

PROF. DR. MRS. L.S. ADMUTHE
(DIRECTOR)

EXAMINER: _____

_____

DECLARATION

We hereby declare that the SRS and Design Document of the mini project titled **"AquaSense"**, submitted to **D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji**, affiliated with **Shivaji University, Kolhapur**, is submitted in partial fulfillment of the requirements for the award of the degree of **Second Year Bachelor of Technology in Computer Science and Engineering**.

This report is a bonafide record of the work carried out by us. The content presented in this report has not been submitted, either in part or in full, to any other university or institution for the award of any degree or diploma.

We further declare that this work complies with all applicable provisions of the **Copyright, Piracy, Cyber Laws, and Intellectual Property Rights (IPR) Acts**, as amended from time to time.

MR. Krish Mahajan	**24UCS056**	**……..**
MR. Shreyash Chougule	**24UCS023**	**……..**
MR. Pranav Chougule	**24UCS021**	**……..**
MR. Varad Dake	**24UCS024**	**……..**

ACKNOWLEDGEMENT

With great pleasure we wish to express our deep sense of gratitude to V.V. Kheradkar for his valuable guidance, support and encouragement in completion of this Mini project report.

Also, we would like to take opportunity to thank our head of department Dr. D. V. Kodavade for his co-operation in preparing this project report.

We feel gratified to record our cordial thanks to other staff members of Computer Science and Engineering Department for their support, help and assistance which they extended as and when required.

Thank you,

MR. Krish Mahajan	24UCS056
MR. Shreyash Chougule	24UCS023
MR. Pranav Chougule	24UCS021
MR. Varad Dake	24UCS024

ABSTRACT

Water scarcity and inefficient usage are critical concerns in shared environments such as hostels, apartments, and institutions where water consumption is rarely monitored in real time. The absence of systematic tracking often leads to tank overflows, unexpected shortages, and excessive wastage, as users lack awareness of how much water is consumed across different areas. AquaSense is proposed as an IoT-based smart water monitoring and management system designed to provide multi-area water intelligence. The system deploys water flow sensors at multiple usage points- including bathrooms, kitchens, drinking water stations, and plantation or utility lines- to independently monitor and analyze section-wise consumption. An ESP32/ESP8266 microcontroller collects and processes real-time flow and tank level data, ensuring that consumption remains aligned with available storage. The analyzed data is displayed on a live mobile or web dashboard, presenting insights such as input flow rate, area-wise output consumption, tank level status, and supply availability. AquaSense further integrates a servo-controlled valve with manual and Auto Mode regulation to optimize distribution, reduce wastage,and promote sustainable water management practices.

INDEX

1. Introduction **1- 4**

a. Problem definition 1

b. Aim and objective of the project 2

c. Scope and limitation of the project 3

d. Timeline of the project 4

e. **Background study** **5-6**

1) Related work 5

2) Investigation of current project and proposed work 6

2. Requirement analysis **7-8**

a. Requirement Gathering 7

b. Requirement Specification 7

1)Functional Requirements

2) Non-Functional Requirements

3)Hardware Requirement

4)Software Requirement 8

3. System design **9-**

a. Architectural Design <<PgNo>>

b. User Interface Design

c. Algorithmic description of each module <<PgNo>>

4.System modelling **<<PgNo>>**

4.1. Structural Design

a. Class Diagrams

b. Object Diagrams

4.2. Data Design

a.Data-flow Diagram

b.ER-Diagram

4.3. Behavioral Design

a. Use case Diagram

b. Sequence Diagram

c. Collaboration diagram

d. Activity Diagram

e. State-chart diagram

4.4. Implementation Design

a. Component Diagram

4.5. Environment Design

a. Deployment Diagram

5. Implementation Details **<<PgNo>>**
a. Environmental Setting for Running the Mini Project <<PgNo>>
b. Detailed Description of Methods <<PgNo>>
c.Implementation Details **<<PgNo>>**

6. Integration and Testing **<<PgNo>>**
a.Description of the Integration Modules
b.Testing
7.Conclusion **<<PgNo>>**
8. References **<<PgNo>>**

1. Introduction

a. Problem Definition

Water scarcity and inefficient water usage have become serious challenges in modern society, especially in shared environments such as hostels, apartments, and educational institutions. In such settings, water is consumed across multiple areas including bathrooms, kitchens, drinking water stations, and utility or plantation zones. However, the absence of proper monitoring systems makes it difficult to track how much water is being used in each area. As a result, water consumption remains largely unregulated and unoptimized.

One of the major issues is the lack of **real-time visibility** into water usage. Users are generally unaware of consumption patterns, which leads to excessive usage in certain areas while other areas may experience shortages. Additionally, storage tanks are often managed manually, which can result in frequent problems such as tank overflows or sudden depletion of water. These issues not only lead to wastage of a valuable resource but also create inconvenience for users.

Another significant limitation is the absence of **area-wise consumption analysis**. Traditional water management systems treat water usage as a single aggregated quantity, without distinguishing between different consumption points. This makes it impossible to identify high-usage zones or detect irregular usage patterns. Consequently, there is no data-driven approach to optimize water distribution or encourage responsible consumption behavior.

Furthermore, water flow control in most systems is manual and inefficient. Users do not have the means to regulate water flow precisely based on actual requirements. This often leads to either excessive flow, causing wastage, or insufficient flow, affecting usability. In shared environments, the lack of coordinated control can also result in unfair distribution of water among users.

With the increasing demand for sustainable resource management, there is a need for a system that not only monitors water usage but also provides actionable insights and intelligent control mechanisms. An effective solution should be capable of tracking water consumption across multiple areas, analyzing usage patterns in real time, and enabling both manual and automated control of water flow.

Therefore, the problem lies in the **absence of an integrated, real-time, and intelligent water monitoring and management system** that can provide area-wise consumption insights, prevent wastage, ensure fair distribution, and support efficient utilization of water resources. Addressing this problem is essential for promoting sustainable practices and improving water management in shared environments.

b. Aim and Objectives of the Project

Aim

The primary aim of the AquaSense project is to develop an IoT-based smart water monitoring and management system that enables efficient, real-time tracking and intelligent control of water usage in shared environments such as homes, hostels, apartments, and institutions. The system aims to transform traditional water usage into a data-driven process by providing clear visibility into consumption patterns, ensuring optimal utilization of available water resources, and reducing wastage through automation and user awareness.

Objectives

To achieve the above aim, the project focuses on the following objectives:

1. **To monitor real-time water consumption across multiple areas**
   The system aims to install flow sensors at different usage points such as bathrooms, kitchens, drinking water stations, and plantation or utility areas in order to track water usage independently in each section.
2. **To measure and track storage tank water levels**
   The project includes monitoring of tank water levels using appropriate sensors, ensuring that water consumption is always aligned with available storage and preventing overflow or depletion.
3. **To analyze area-wise water consumption patterns**
   The system is designed to process collected data and identify high-consumption zones, irregular usage patterns, and inefficiencies in water distribution.
4. **To provide intelligent suggestions for efficient water usage**
   Based on real-time analysis, the system aims to generate actionable insights and recommendations that help users optimize water consumption and reduce wastage.
5. **To implement a smart flow control mechanism**
   The project integrates a servo-controlled valve that allows regulation of water flow. Users can manually control the flow through a dashboard or enable Auto Mode for automatic adjustment based on system analysis.
6. **To develop a user-friendly dashboard interface**
   A mobile or web-based dashboard is developed to display real-time information such as input flow rate, area-wise consumption, tank level status, and system suggestions, ensuring easy interaction and monitoring.
7. **To enable automation and remote accessibility**
   The system utilizes IoT technology to allow remote monitoring and control, providing users with flexibility and convenience in managing water usage from anywhere.
8. **To promote sustainable and responsible water management practices**
   The overall objective is to create awareness among users and encourage efficient water usage by providing data-driven insights and control mechanisms.

c. Scope and Limitations of the Project

Scope of the Project

The AquaSense project focuses on developing an IoT-based smart water monitoring and management system that enables efficient utilization of water resources through real-time tracking, analysis, and control. The system is designed to monitor water consumption across multiple usage areas such as bathrooms, kitchens, drinking water stations, and plantation or utility lines. By installing flow sensors at different points, the system provides detailed, area-wise insights into water usage, allowing users to identify high-consumption zones and optimize their usage accordingly.

The project also includes monitoring of storage tank water levels, ensuring that water consumption is always aligned with available supply. The integration of a microcontroller (ESP32/ESP8266) allows real-time data processing and communication with a mobile or web-based dashboard. This dashboard enables users to view important parameters such as input flow rate, output consumption, tank level status, and supply availability from anywhere.

Another important aspect of the project is the implementation of a smart control mechanism using a servo-operated valve. This feature allows both manual and automatic regulation of water flow, helping to reduce wastage and ensure fair distribution of water across different areas. The system also provides intelligent suggestions based on consumption patterns, promoting efficient and sustainable water usage.

The solution is scalable and can be extended to various environments including homes, hostels, apartments, institutions, and small-scale industries. It has the potential to be integrated with advanced technologies such as cloud storage, data analytics, and smart city infrastructure for larger applications.

Limitations of the Project

Despite its advantages, the AquaSense system has certain limitations. The accuracy of the system depends on the quality and calibration of sensors used. Low-cost flow sensors and level sensors may introduce minor measurement errors, which can affect the precision of the data.

The system also relies on continuous Wi-Fi connectivity for real-time data transmission and dashboard updates. Any interruption in network connectivity may temporarily affect monitoring and control features. Additionally, the implementation requires proper installation of sensors at multiple points, which may increase initial setup complexity and cost.

Another limitation is that the system is primarily designed for small to medium-scale environments and may require further enhancements for large-scale industrial applications. Environmental factors such as water pressure variations, pipe conditions, and sensor wear and tear over time may also impact system performance.

Furthermore, while the system provides suggestions and automation, user behavior still plays a significant role in effective water conservation. Complete optimization cannot be achieved without user awareness and cooperation.

d. Timeline of the Project

The AquaSense project, in its current phase, focuses on the software development and simulation of a smart water monitoring and management system. The development process was carried out in a structured and phased manner as follows:

Phase 1: Problem Identification and Concept Finalization (Week 1–2)

The project began with identifying issues related to water wastage and lack of monitoring in shared environments. The concept of a multi-area water monitoring system with a smart dashboard was finalized.

Phase 2: Requirement Analysis and Research (Week 3–4)

Research was conducted on IoT platforms, dashboard systems, and sensor-based data processing. Functional and non-functional requirements were defined for the system.

Phase 3: System Design and Planning (Week 5–6)

The system architecture was designed, including data flow from sensors to the dashboard. Logic for features such as area-wise monitoring, suggestion generation, and Auto Mode was planned.

Phase 4: Dashboard Development (Week 7–9)

A mobile/web-based dashboard (Blynk/web interface) was developed to display parameters such as flow rates, tank level, and system status. UI design focused on clarity and real-time visualization.

Phase 5: Algorithm and Logic Implementation (Week 10–12)

Core system logic was implemented, including:

- Water usage analysis
- Suggestion generation
- Auto Mode functionality
- Manual control options

Phase 6: Testing and Validation (Week 13–14)

The system was tested using simulated or real-time data to verify accuracy, responsiveness, and functionality of features like suggestions and automation.

Phase 7: Documentation and Finalization (Week 15–16)

Final documentation, report preparation, abstract submission, and presentation (PPT) were completed.

e. Background Study

1) Related Work

Water management has been an important area of research due to increasing water scarcity and inefficient usage. Traditional water management systems mainly rely on manual monitoring and centralized storage, where water usage is not tracked at individual consumption points. This often leads to wastage, uneven distribution, and lack of awareness among users.

Several modern systems have introduced the use of IoT for water monitoring. These systems typically use sensors to measure parameters such as water flow, tank levels, and leakage detection. Some solutions provide real-time monitoring through dashboards and mobile applications, enabling users to track water usage remotely. However, most existing systems focus only on overall water consumption and do not provide detailed insights into area-wise usage.

In addition, many current solutions lack intelligent control mechanisms. While they may display data, they do not actively assist users in optimizing water usage. Features such as automated flow regulation, smart suggestions, and multi-area monitoring are either limited or not integrated effectively. This creates a gap between monitoring and actual water management.

2) Investigation of Current System

The current water usage systems in homes, hostels, and institutions are largely manual and lack real-time monitoring capabilities. Water is supplied through a centralized system without any mechanism to track consumption at different usage points such as kitchens, bathrooms, or utility areas.

Key limitations of the current system include:

- **No area-wise monitoring:** Water consumption is measured as a whole, without identifying which area uses more water.
- **Lack of real-time data:** Users do not have access to live information about water usage or tank levels.
- **Manual control:** Water flow is regulated manually, leading to inefficiency and wastage.
- **No intelligent insights:** There is no system to analyze consumption patterns or provide suggestions.
- **Inefficient resource utilization:** Due to lack of awareness and control, water is often wasted or unevenly distributed.

These limitations highlight the need for a more advanced and intelligent water management system.

3) Proposed Work

To overcome the limitations of existing systems, AquaSense is proposed as a smart, IoT-based water monitoring and management solution with a focus on multi-area consumption tracking and intelligent control.

The proposed system introduces the following improvements:

- **Multi-area monitoring:** Water flow sensors are placed at different usage points such as bathrooms, kitchens, drinking areas, and plantation lines to track consumption independently.
- **Real-time data processing:** A microcontroller (ESP32/ESP8266) processes sensor data and provides continuous monitoring of flow and tank levels.
- **Interactive dashboard:** A mobile/web-based dashboard displays key parameters including input flow, area-wise consumption, tank level, and supply status.
- **Smart suggestions:** The system analyzes usage patterns and provides recommendations to optimize water usage.
- **Automatic flow regulation:** A servo-controlled valve enables both manual control and Auto Mode for efficient water distribution.

The proposed system aims to bridge the gap between monitoring and management by combining real-time data, intelligent insights, and automated control. It not only helps reduce water wastage but also promotes awareness and sustainable usage practices.

2. Requirement Analysis

a. Requirement Gathering

Requirement gathering is the initial and one of the most important phases in the development of the AquaSense system. In this phase, the needs and expectations of users were identified to ensure that the system effectively addresses real-world problems related to water management.

The requirements were gathered by analyzing common issues faced in shared environments such as hostels, apartments, and institutions. Observations were made regarding water wastage, lack of monitoring, uneven distribution, and absence of real-time information about water usage. Discussions with potential users also helped in understanding the practical challenges in managing water efficiently.

Based on this analysis, it was identified that users require a system that can monitor water consumption in real time, provide insights into usage across different areas, and offer control mechanisms to regulate water flow. Since the project currently focuses on software implementation, emphasis was placed on features such as data visualization, system logic, and user interaction through a dashboard.

The key requirements gathered for the system are as follows:

- The system should display **real-time water usage data**.
- It should support **multi-area monitoring** (kitchen, bathroom, etc.).
- It should show **tank level and supply status**.
- It should provide **area-wise consumption insights**.
- It should generate **smart suggestions** to reduce water wastage.
- It should allow **manual control options** for flow regulation.
- It should include an **Auto Mode** for automatic decision-making.
- The dashboard should be **simple, user-friendly, and accessible remotely**.

Thus, the requirement gathering phase ensured that the AquaSense system is designed to meet user needs effectively while focusing on practical implementation and usability.

b. Requirement Specification

Requirement specification defines the detailed functional and non-functional needs of the AquaSense system, along with the hardware and software components required for its implementation. These specifications ensure that the system is developed in a structured and efficient manner.

1) Functional Requirements

Functional requirements describe the core operations that the system must perform.

- The system shall monitor water flow in real time using flow sensors.
- The system shall support multi-area monitoring (bathroom, kitchen, drinking water, plantation, etc.).
- The system shall measure and display tank water levels.

- The system shall process collected data to analyze water usage patterns.
- The system shall display all parameters on a mobile/web-based dashboard.
- The system shall provide area-wise consumption insights.
- The system shall generate smart suggestions for reducing water wastage.
- The system shall allow manual control of water flow through a dashboard interface.
- The system shall support Auto Mode for automatic flow regulation.
- The system shall indicate water supply availability.

2) Non-Functional Requirements

Non-functional requirements define the performance and quality aspects of the system.

- The system should provide real-time updates with minimal delay.
- The system should be reliable and stable during continuous operation.
- The dashboard should be user-friendly and easy to understand.
- The system should be scalable to support additional monitoring areas.
- The system should ensure data accuracy with minimal errors.
- The system should consume low power for efficient operation.
- The system should be secure, allowing access only to authorized users.
- The system should be compatible with mobile and web platforms.

3) Hardware Requirements

- ESP8266 / ESP32 Microcontroller
- Water Flow Sensors (e.g., YF-S201)
- Ultrasonic Sensor (HC-SR04) for tank level measurement
- Servo Motor for valve control
- Connecting wires and power supply
- Wi-Fi module (built-in in ESP8266/ESP32)

4) Software Requirements

- Programming Language: C/C++ (Arduino IDE)
- IoT Platform: Blynk / Web Dashboard
- Development Environment: Arduino IDE / VS Code
- Simulation Tools (optional): Proteus / Tinkercad / Wokwi
- Operating System: Windows / macOS / Linux
- Web Technologies (if extended): HTML, CSS, JavaScript

3. System design

A. Architecture Design

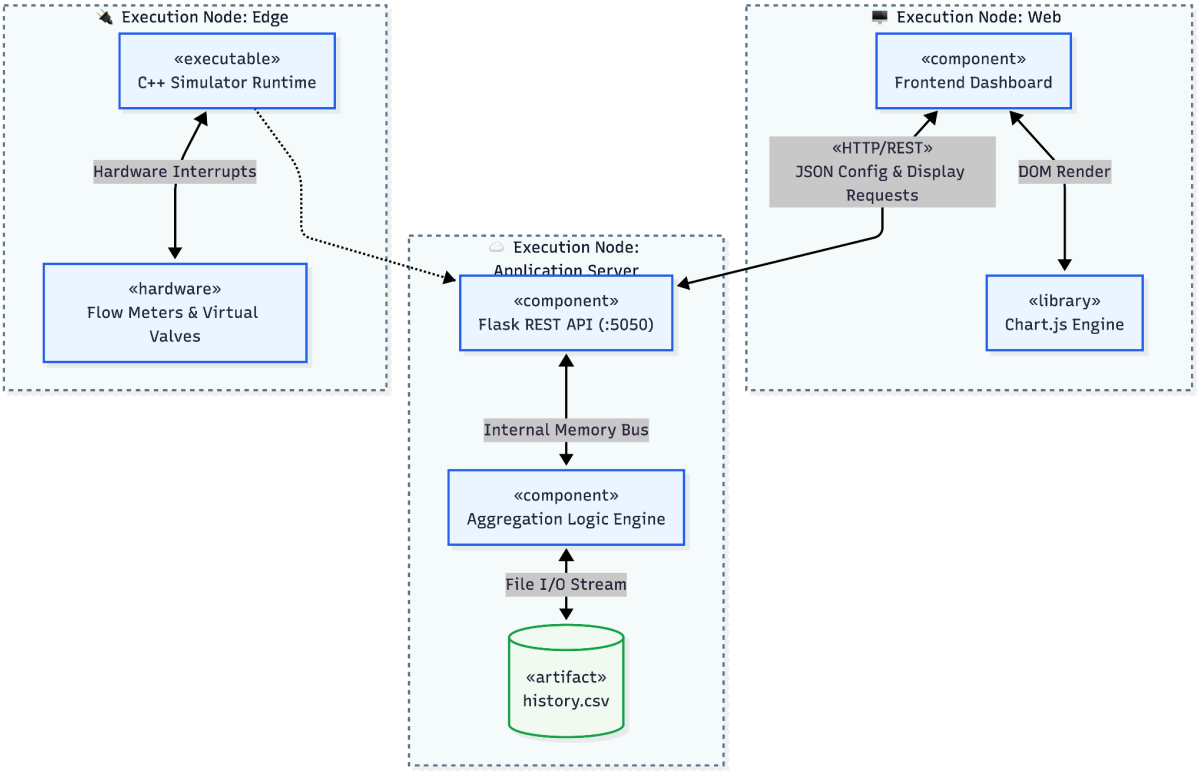


Fig 3.1

Description:

Architecture Diagram (Fig 3.1) Provides a high-level overview of the AquaSense system's structure, illustrating the separation of concerns across the Presentation, Application, Data, and Hardware layers, alongside the data flow between them.

B. User Interface Design

C. Algorithmic description of each module

4.System modelling

4.1. Structural Design

4.1.a. Class Diagrams

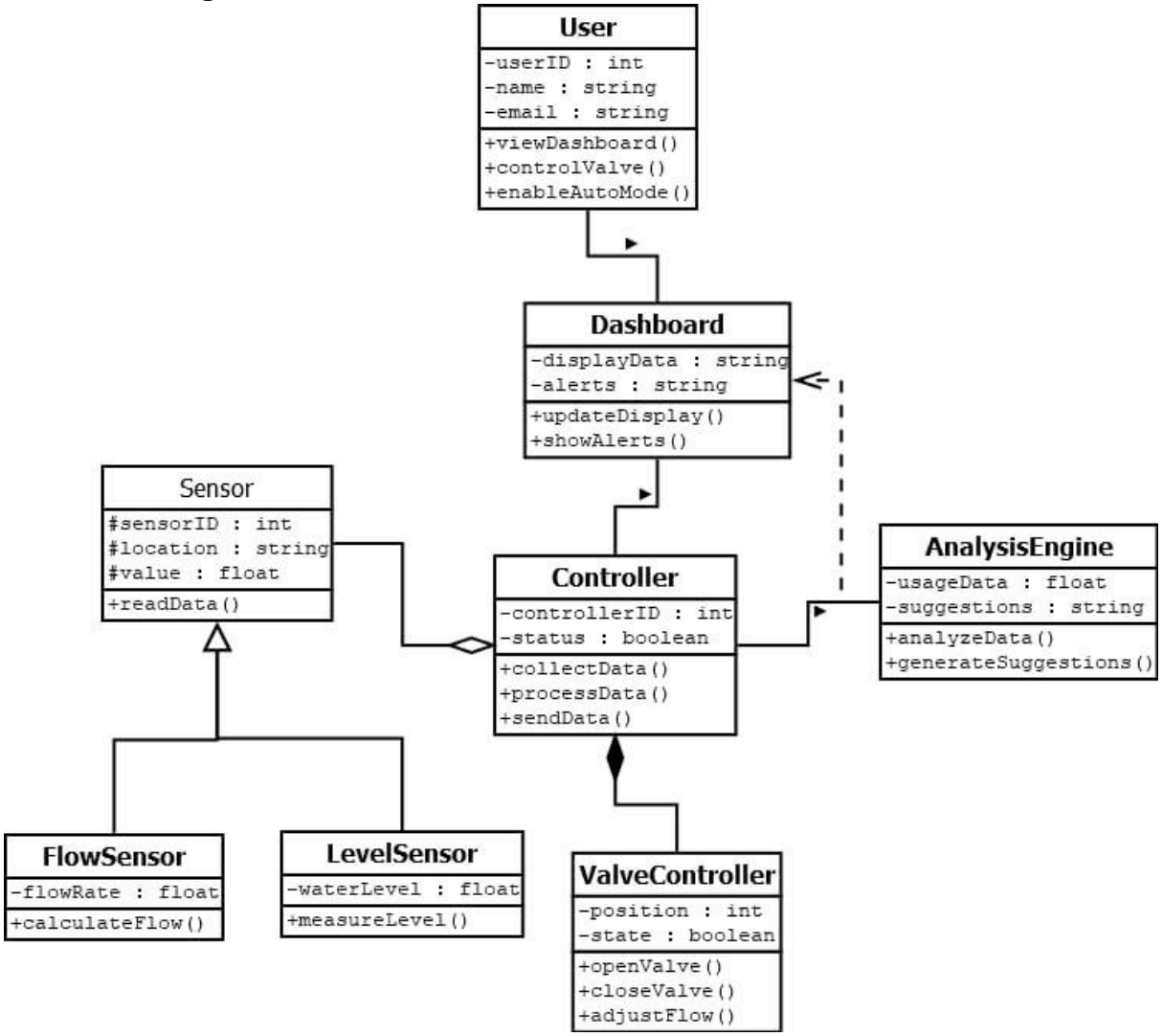


Fig. 4.1.a

Description:
Class Diagram (Fig. 4.1.a) Illustrates the static object-oriented structure of the system's software, detailing core classes (e.g., SensorManager, AnalyticsEngine), their attributes, methods, and inheritances.

4.1.b. Object Diagrams

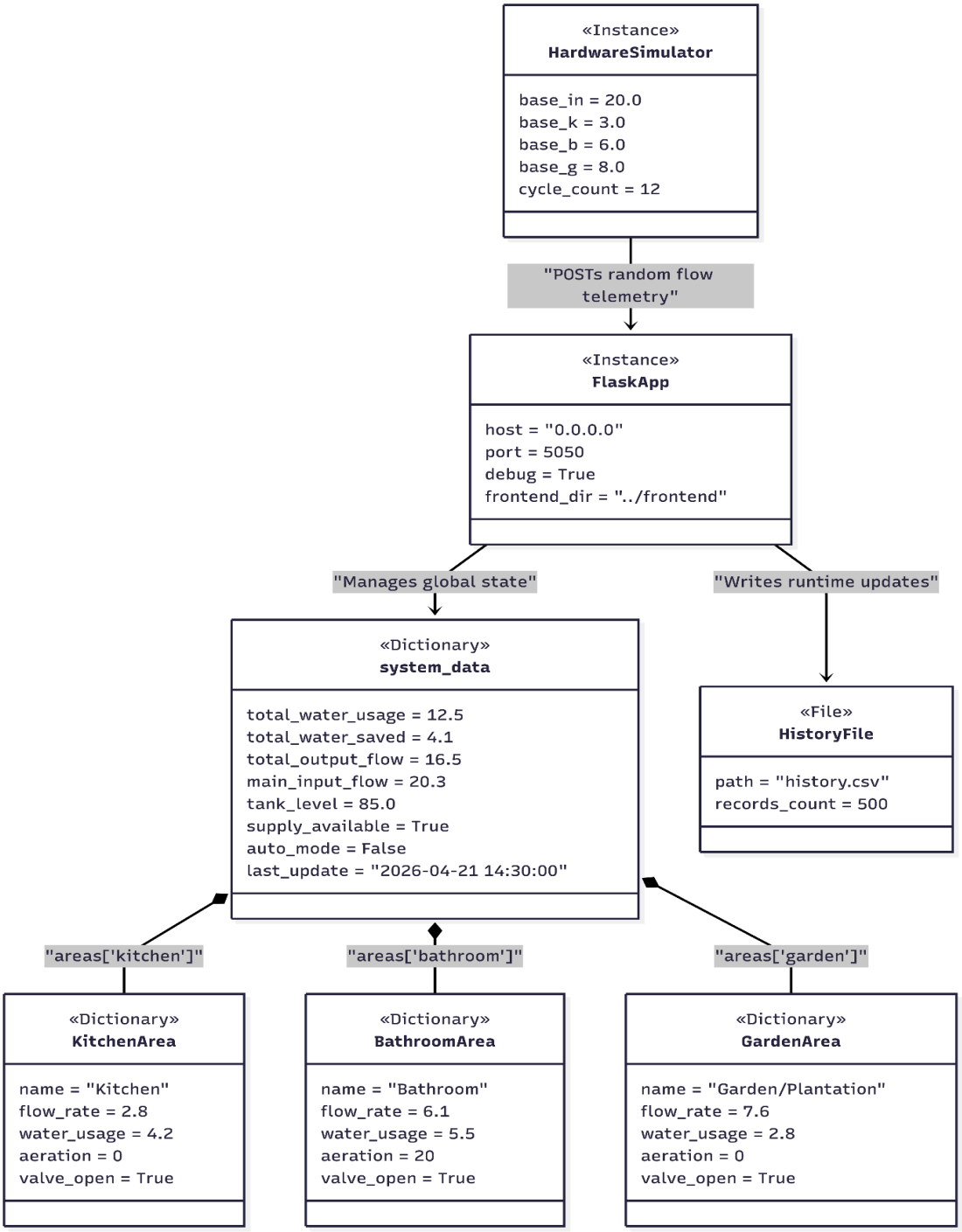


Fig. 4.1.b

Description:
Object Diagram (Fig. 4.1.b) Provides a static snapshot of the system at a specific moment in time, displaying active instances of classes and their exact state values (e.g., current tank level at 80%, kitchen valve open).

4.2. Data Design

4.1.a.Data-flow Diagram

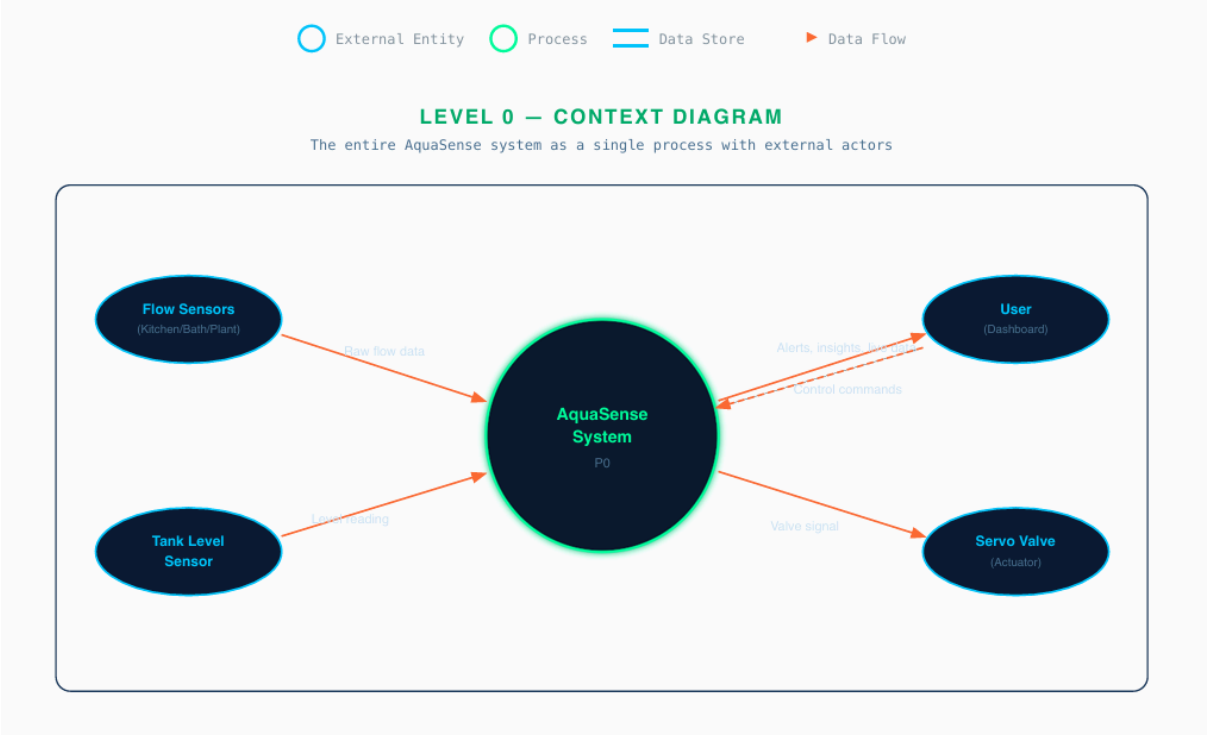


Fig. 4.2.a.i

Description for Fig. 4.2.a.i: Level 0 Context Diagram :

Figure 4.2.a.i illustrates the Context Diagram (Level 0 DFD) for the AquaSense Smart Water Management System. This diagram defines the boundaries of the system and highlights the flow of information between the core application and its four primary external entities:

- Flow Sensors: These provide real-time input data regarding the rate of water movement through the pipes, allowing the system to monitor consumption.
- Tank Level Sensor: This entity sends continuous data to the system regarding the current water depth or volume in the storage tanks to prevent overflow or dry-running.
- User Dashboard: This is the primary interface where the system outputs processed data (such as alerts, usage statistics, and system status) and receives manual overrides or configuration settings from the user.
- Servo Valve (Actuator): Based on the logic processed within the AquaSense system (e.g., tank full or leak detected), the system sends control signals to this entity to physically open or close the water supply.

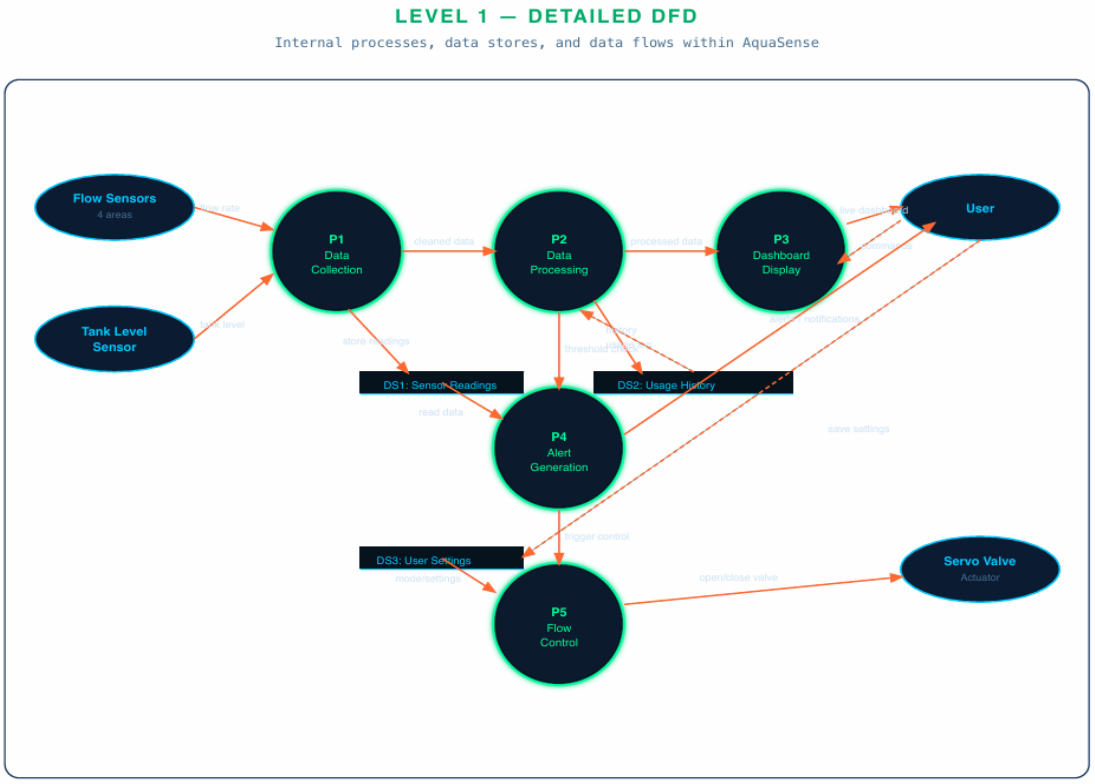


Fig. 4.2.a.ii

Description for Fig. 4.2.a.ii: Level 1 Data Flow Diagram :

Figure 4.2.a.ii presents the Level 1 Data Flow Diagram, which provides a more detailed view of the internal functional modules of the AquaSense system. This diagram decomposes the "Smart Water Management" process into several sub-processes to show how data is transformed as it moves through the system:

- Process 1.0: Sensor Data Acquisition: This process acts as the interface for the Flow and Tank Level sensors. It captures raw signals and converts them into digital values for consumption and water level.
- Process 2.0: Data Processing & Analysis: The heart of the system logic. Here, the acquired data is compared against predefined thresholds (e.g., "Is the tank full?" or "Is there an unusual flow indicating a leak?").
- Process 3.0: Actuator Control: Based on the analysis, this process generates commands for the Servo Valve. If the tank is full or a leak is detected, it triggers the closing of the valve.
- Process 4.0: Dashboard Management: This process handles the communication with the User Dashboard, updating the UI with real-time stats and processing user input commands for manual system overrides.
- Data Stores: The diagram also illustrates how information is saved to the system's database (or logs) for historical tracking and future analysis of water usage patterns.

4.2.b. ER-Diagram

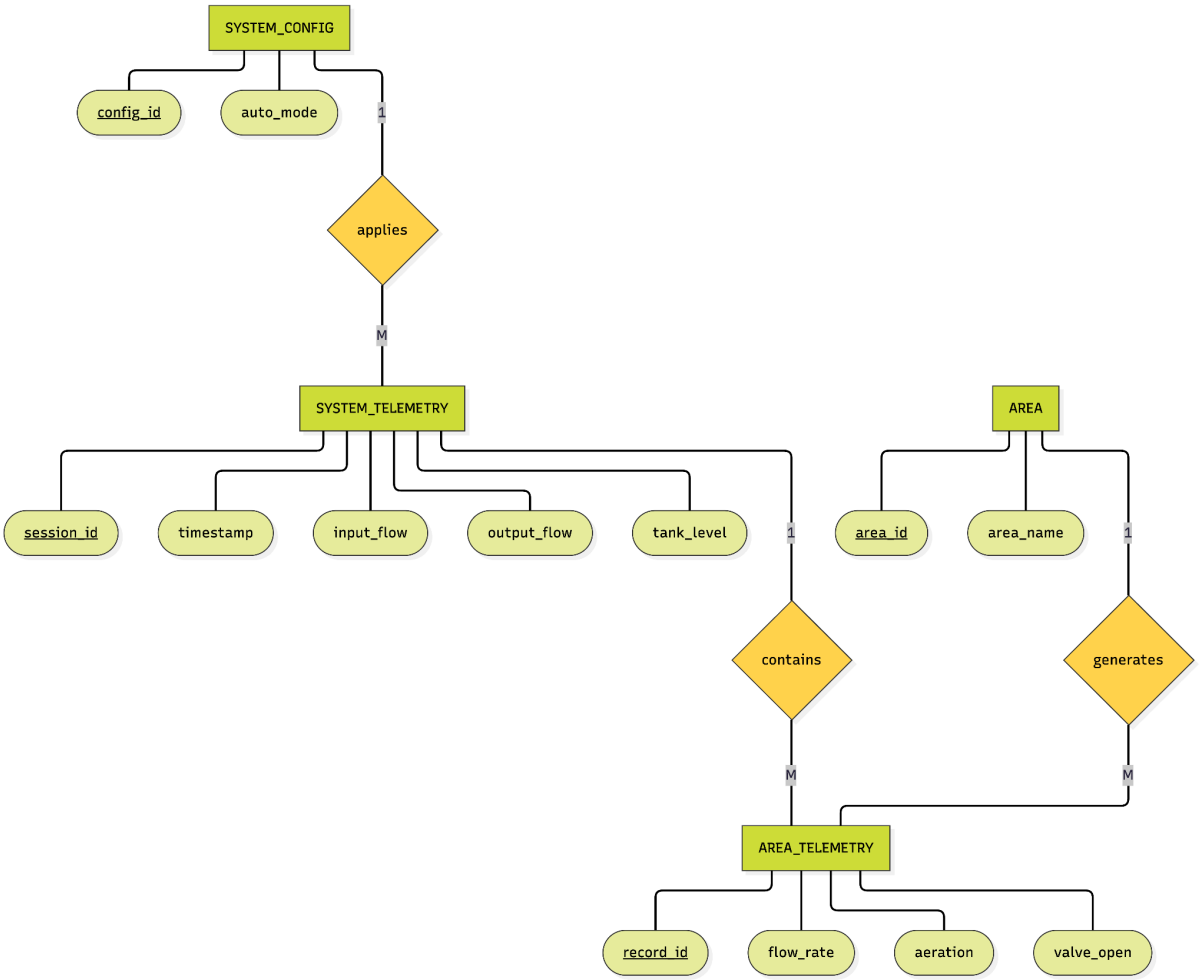


Fig. 4.2.b

Description:

Entity-Relationship (ER) Diagram (Fig. 4.2.b) Defines the conceptual database schema, showcasing entities such as System Telemetry, Area-Specific Data, and Configurations, along with their attributes and relational mappings.

— — — P.T.O

4.3. Behavioral Design

4.3.a. Use case Diagram

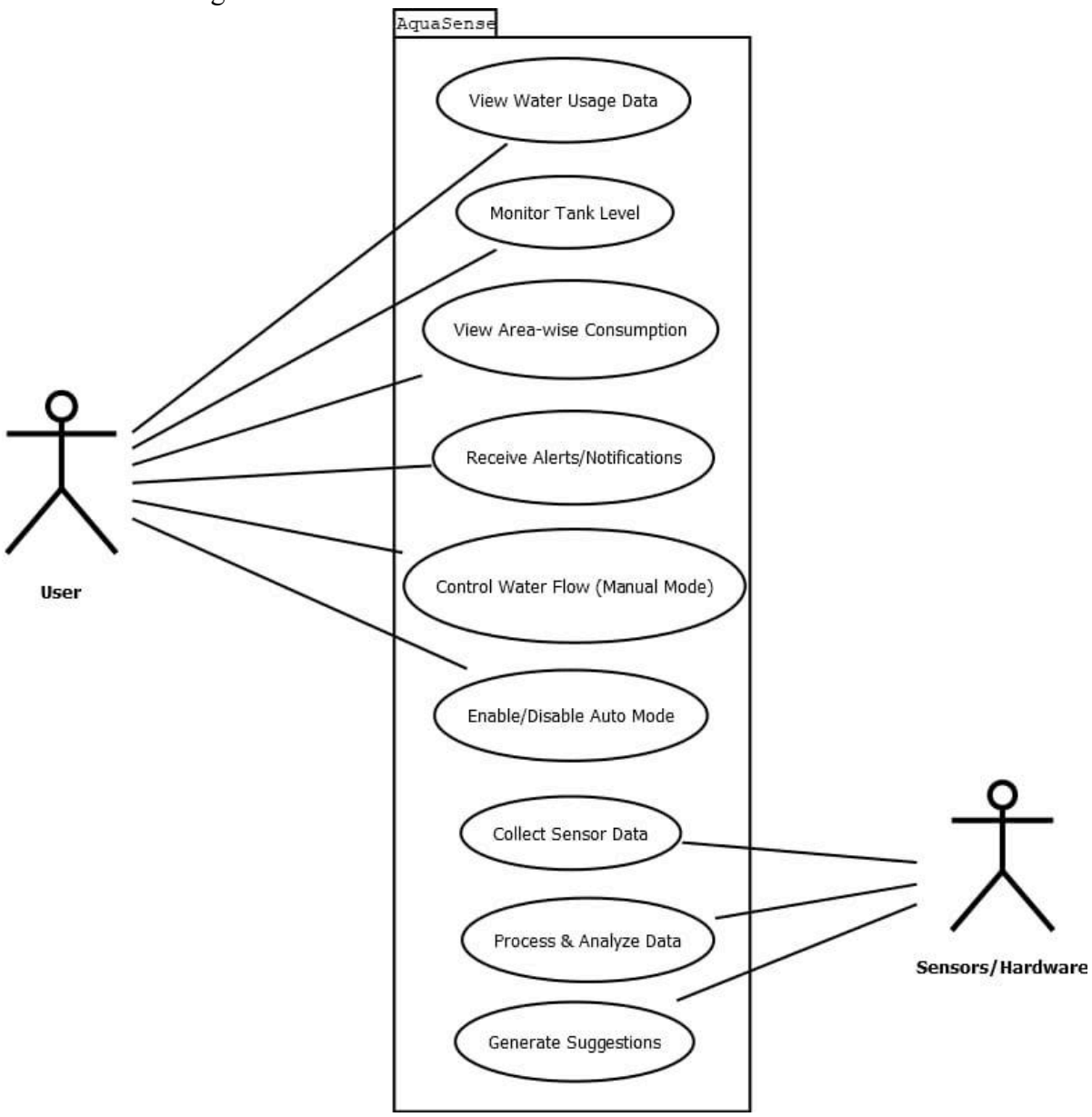


Fig. 4.3.a

Description:

Use Case Diagram (Fig. 4.3.a) Outlines the system from an external perspective, highlighting the interactions between actors (User, Admin, Hardware Sensors) and the core functional goals they can achieve within AquaSense.

4.3.b. Sequence Diagram

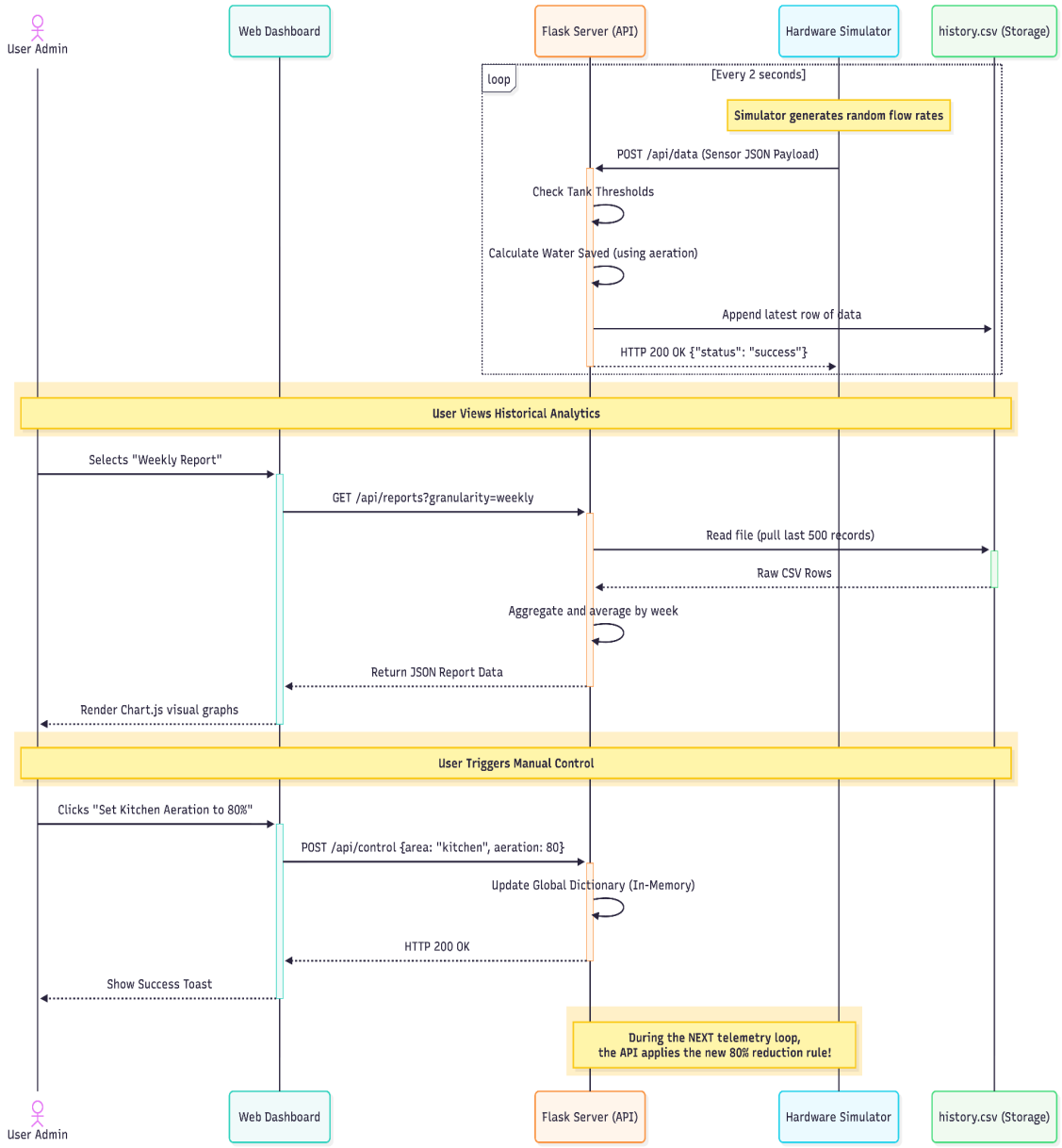


Fig. 4.3.b

Description:

Sequence Diagram (Fig. 4.3.b) Details the chronological order of messages and interactions between system actors and components over time, such as the step-by-step process of a user requesting and receiving an analytics report.

4.3.c. Collaboration diagram

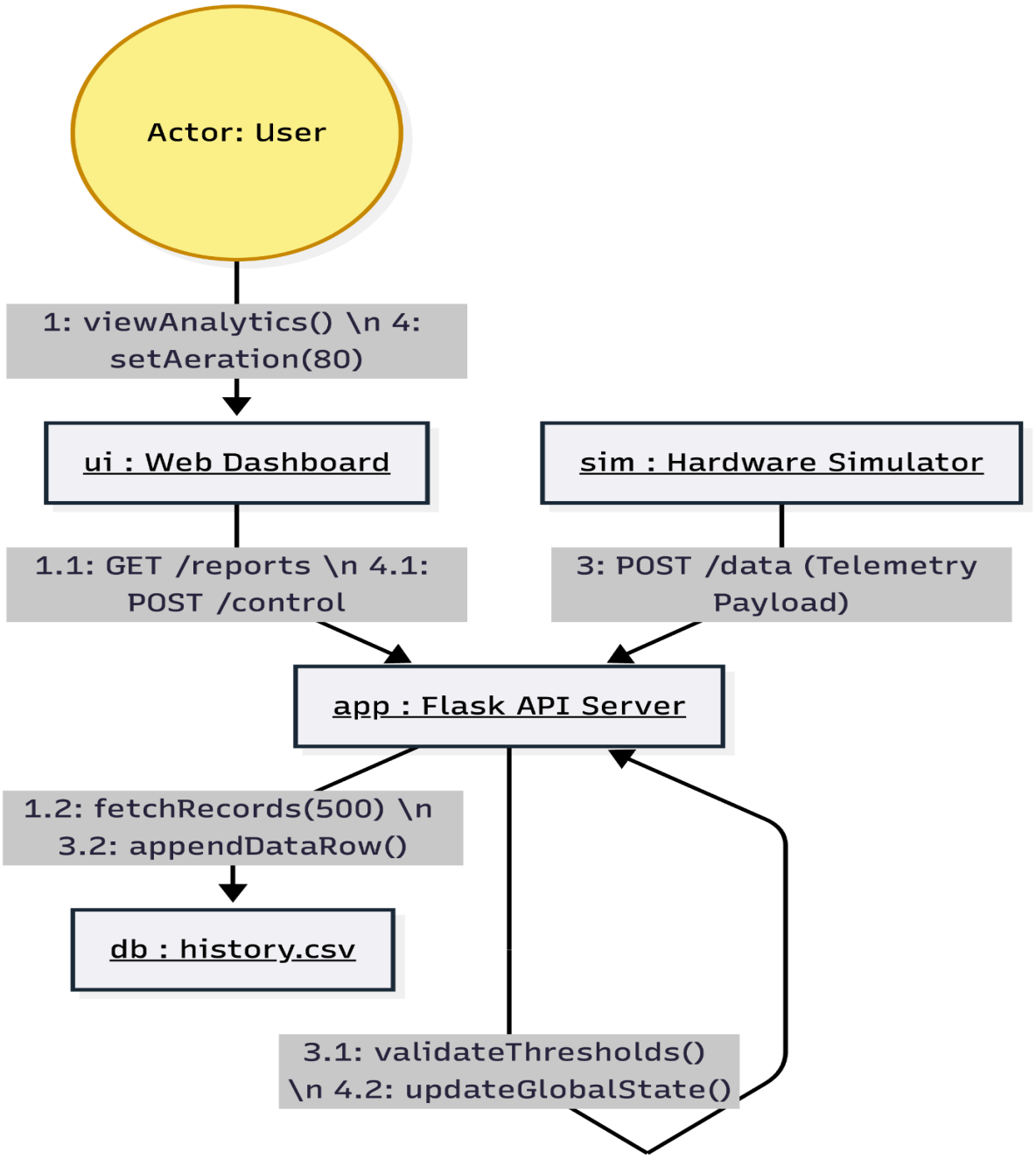
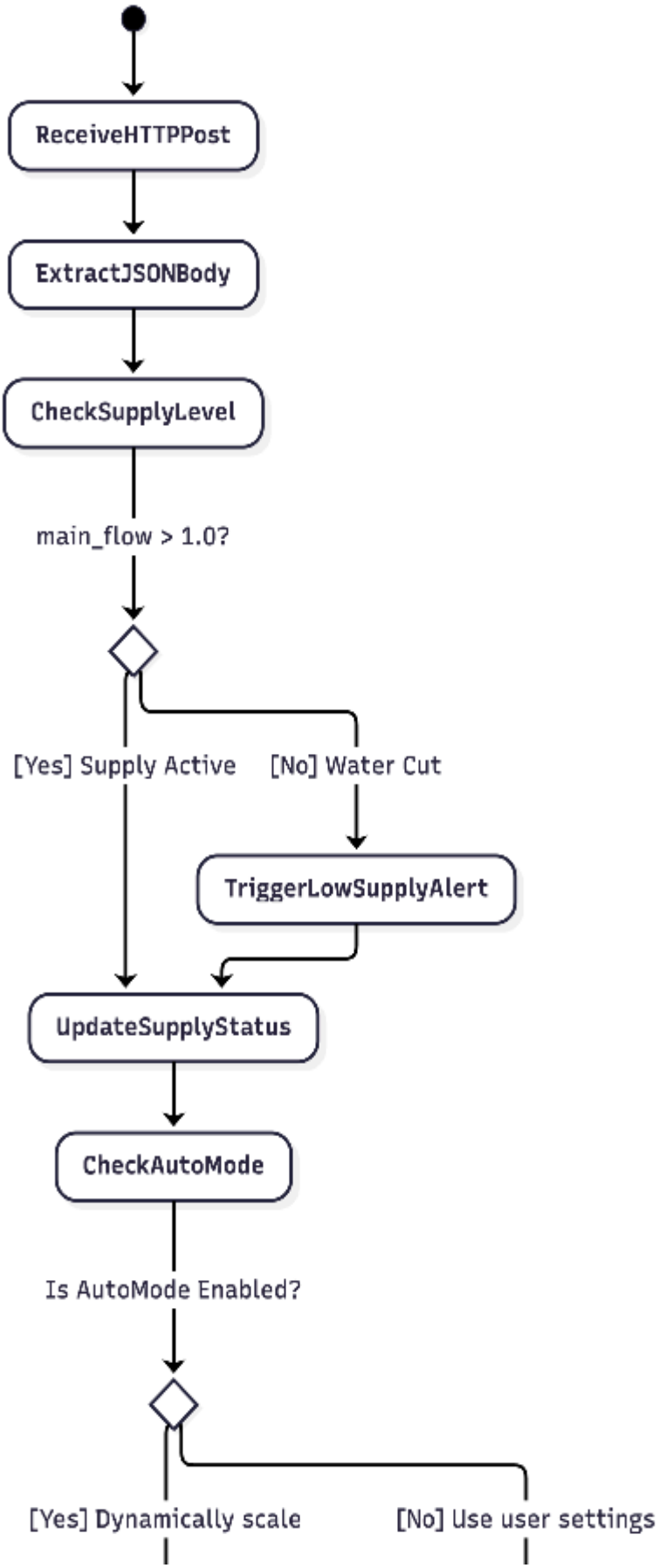
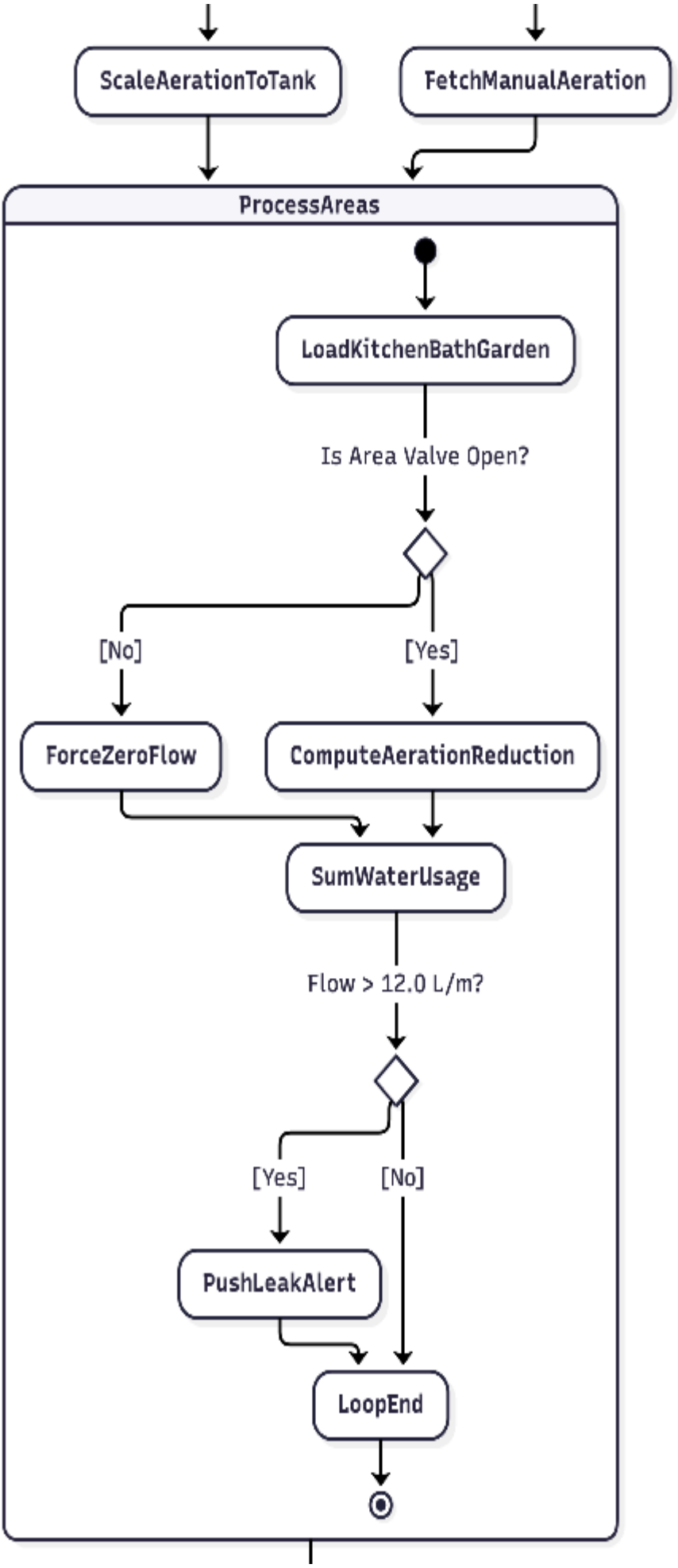


Fig. 4.3.c

Description:
Collaboration Diagram (Fig. 4.3.c) Focuses on the structural organization of objects that send and receive messages, highlighting how specific components (e.g., UI, REST API, Database) collaborate to fulfill telemetry processing.

4.3.d. Activity Diagram




ScaleAerationToTank
FetchManualAeration
ProcessAreas
LoadKitchenBathGarden
Is Area Valve Open?
[No]
[Yes]
ForceZeroFlow
ComputeAerationReduction
SumWaterUsage
Flow > 12.0 L/m?
[Yes]
[No]
PushLeakAlert
LoopEnd

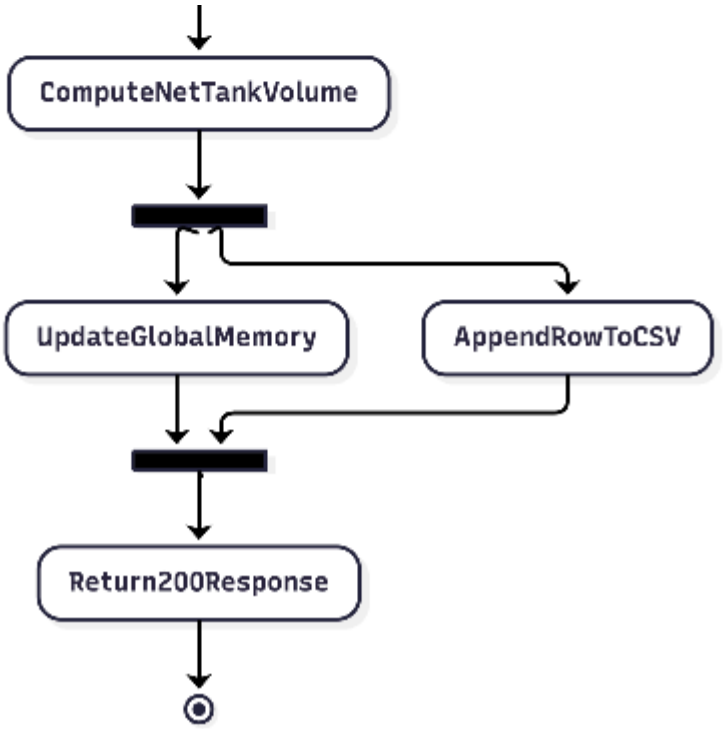


Fig. 4.3.d

Description:

Activity Diagram (Fig. 4.3.d) Maps the dynamic flow of operations and decision-making processes within the system, from real-time sensor data collection and threshold evaluation to executing automated control commands.

4.3.e. State-chart diagram

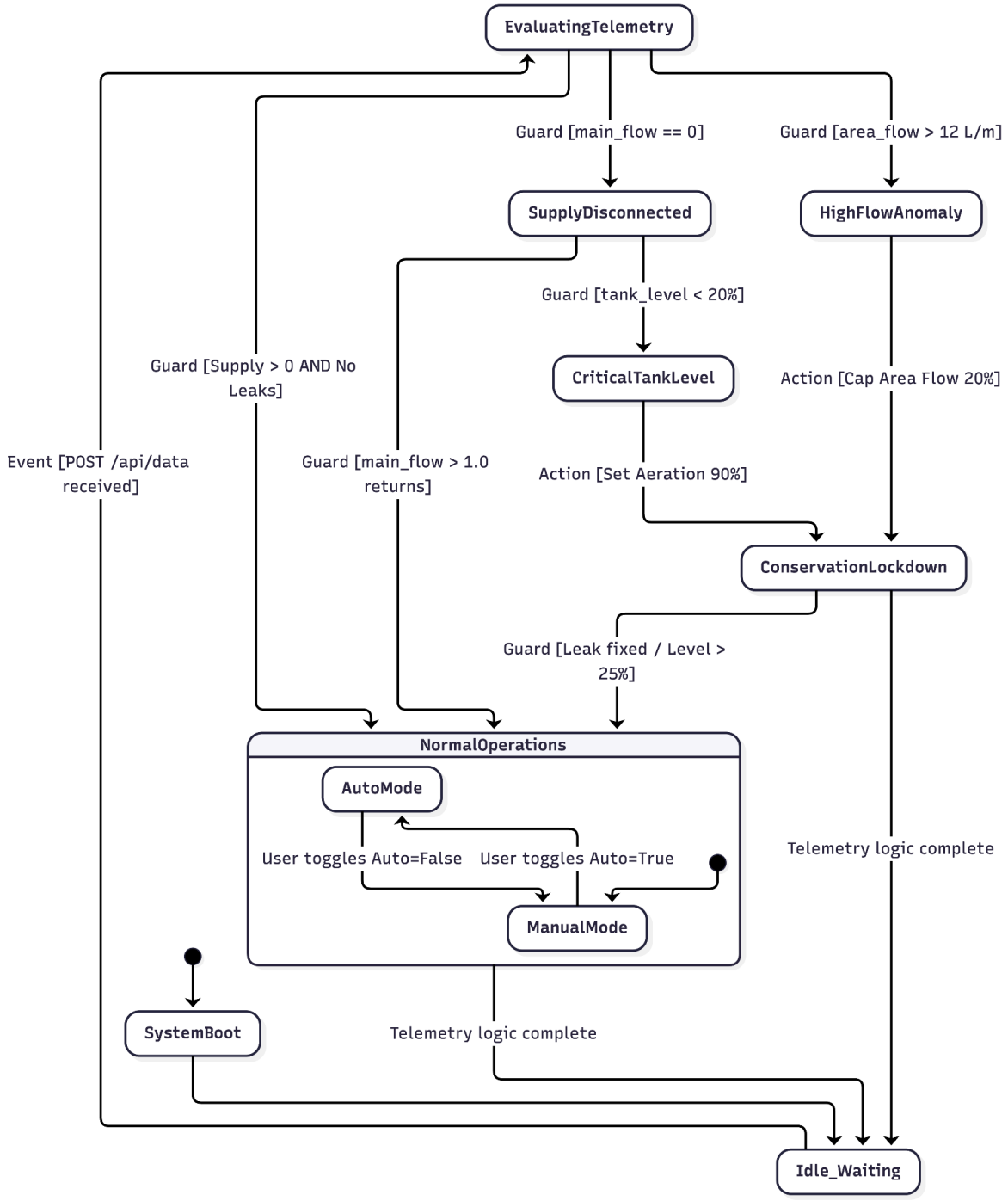


Fig. 4.3.e

Description:
State-chart Diagram (Fig. 4.3.e) Represents the various states that critical system components (like the main water tank or control valves) can exist in, and the specific events or triggers that cause transitions between these states (e.g., Filling -> Full -> Draining).

4.4. Implementation Design

— — — P.T.O

4.4.a Component Diagram

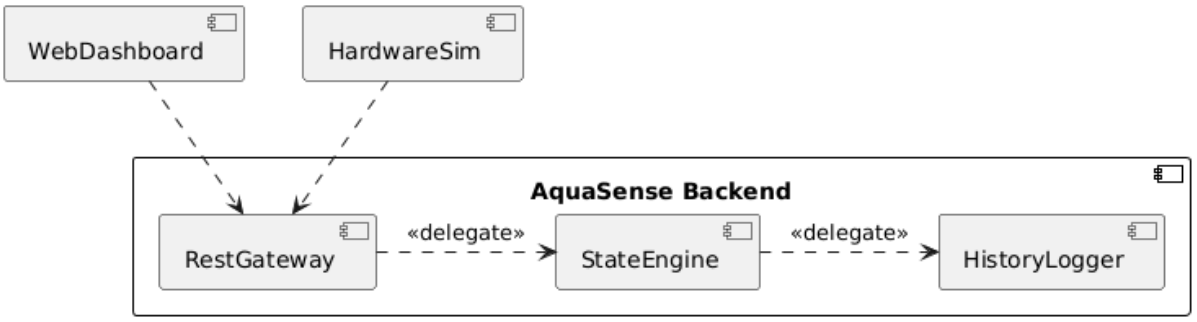


Fig. 4.4.a

Description:
Component Diagram (Fig. 4.4.a) Depicts the modular software and hardware components (e.g., Flask REST API, JavaScript Frontend, C++ Simulator) and the communication interfaces and dependencies connecting them.

4.5. Environment Design

4.5.a. Deployment Diagram

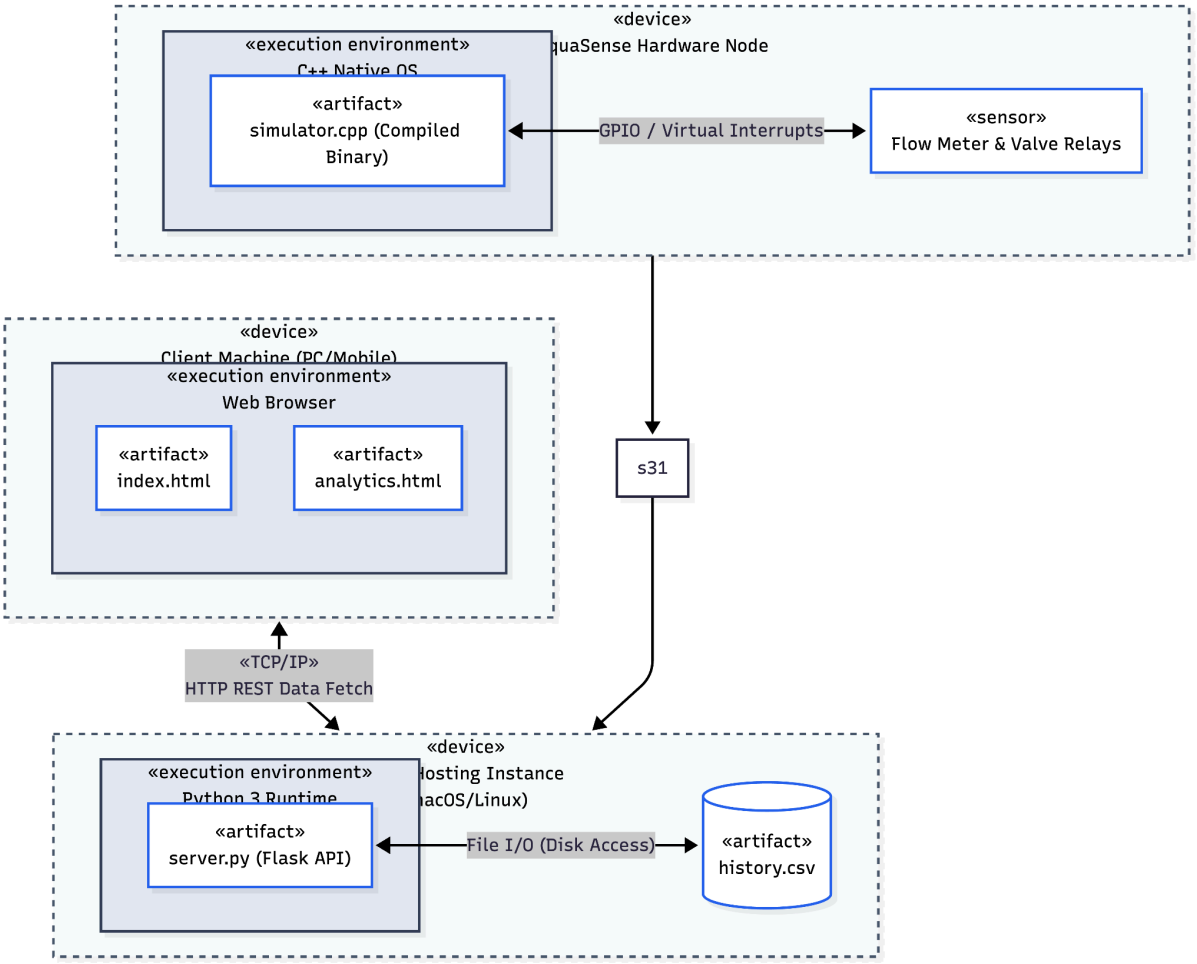


Fig. 4.5.a

Description:

Deployment Diagram (Fig. 4.5.a) Maps the software architecture onto the physical hardware execution environment, illustrating the physical nodes (web servers, client devices, IoT microcontrollers) and the network protocols linking them.

5. Implementation Details

5.a. Environmental Setting for Running the Mini Project

To successfully compile, execute, and evaluate the AquaSense system, the following hardware and software environmental settings are required:

1. Hardware Environment:

Server Node: A standard PC, laptop, or Raspberry Pi (minimum 2GB RAM, 1.5 GHz Processor) to host the central backend server and execute the C++ hardware simulator.
Client Node: Any device (Laptop, Tablet, or Smartphone) connected to the same Local Area Network (LAN) or Wi-Fi network as the Server Node.
Sensors/Actuators (Simulated): In lieu of physical flow meters and solenoid valves, processing power is allocated to the C++ runtime to simulate real-time GPIO hardware interrupts.

2. Software Environment:

Operating System: Cross-platform compatibility (Windows 10/11, macOS, or Linux/Ubuntu).
Backend Environment: Python 3.8 or higher.
Dependencies: `Flask` framework for REST API hosting, standard libraries (`csv`, `socket`, `datetime`).
Hardware Simulator Environment: A standard C++ compiler (GCC, MinGW, or Clang) supporting C++11 or higher.
Frontend Environment: A modern web browser (Google Chrome, Mozilla Firefox, or Safari).
Dependencies: `Chart.js` library (loaded via CDN) for rendering interactive data visualizations.
Network Constraints: The environment requires the host machine to bind to `0.0.0.0` on port `5050` to allow external LAN devices to access the dashboard.

5.b. Detailed Description of Methods

The AquaSense project utilizes several core programmatic methods to achieve real-time telemetry processing, automated conservation, and data visualization:

1. Telemetry Aggregation & Parsing Method:

Data from the hardware layer is continuously pushed to the server via HTTP POST requests containing JSON payloads. The backend utilizes a continuous parsing method that extracts individual node flow rates (Kitchen, Bathroom, Garden). This data is immediately validated against tank capacity constraints before being loaded into a centralized, in-memory global dictionary to allow for instantaneous read access by the frontend dashboard.

2. Adaptive Conservation (Auto Mode) Algorithm:

To ensure water sustainability, the system implements a dynamic scaling method. When the "Auto Mode" flag is true, the algorithm monitors the `tank_level` variable. If the tank volume drops below defined critical thresholds (e.g., 25% or 10%), the method programmatically overrides user settings and forces a higher aeration percentage. The method then recalculates the *Effective Flow* by mathematically reducing the raw sensor input by the applied aeration percentage, logging the difference as `WaterSaved`.

3. Anomaly Detection Method:

The backend utilizes a bounded threshold detection method to identify potential leaks or burst pipes. During the telemetry loop, if any specific area's raw flow rate exceeds 12.0 L/min while valves are expected to be nominal, the method triggers a high-priority system alert and automatically throttles the specific area's flow to prevent flooding.

4. Time-Series Data Aggregation Method:

For historical analytics, the system uses a persistent storage method. A subset of the live telemetry is appended sequentially to a `history.csv` file. When the user requests a report, a specialized aggregation method reads the last 500 records, groups the timestamps based on the requested granularity (Hourly, Daily, or Weekly), and computes the mathematical mean (average) for input flow, output flow, and tank levels over that period.

5.c. Implementation Details

The physical construction and coding of the AquaSense project is divided into three distinct implementation tiers:

1. Backend Implementation (Application Tier):

The core routing and logic engine was implemented using the **Python Flask** framework.

The API was structured around stateless REST architecture. The `@app.route('/api/data')` endpoint was implemented to handle bi-directional logic: it processes `GET` requests to serve live JSON state data to the dashboard, and `POST` requests to ingest data from the hardware simulator.

Persistent storage was implemented using Python's native `csv.writer` module, ensuring that critical telemetry and anomaly events are permanently logged to the hard disk

without requiring a heavy SQL database overlay.

2. Frontend Implementation (Presentation Tier):

The user dashboard was implemented using pure **HTML, CSS, and Vanilla JavaScript** to ensure lightweight execution.

Asynchronous Polling: JavaScript `fetch()` APIs were implemented to poll the Flask server every few seconds. This allows the dashboard metrics and progress rings to update live without requiring a page refresh.

Data Visualization: The `Chart.js` engine was implemented to bind directly to the `/api/reports` endpoint. It dynamically maps the aggregated CSV JSON arrays onto multi-line, responsive HTML5 Canvas charts.

3. Hardware Simulation Implementation (Edge Tier):

Because physical IoT flow-meters were unavailable for the initial environment, the hardware edge nodes were implemented via a **C++ Runtime Engine** (`simulator.cpp`).

The script uses `<random>` and `<chrono>` libraries to generate pseudo-random baseline flow rates for three separate zones, simulating organic human water usage patterns.

It implements a deterministic loop that purposefully cuts the municipal main supply (`main_flow = 0.0`) at predetermined intervals to test the backend's emergency mitigation logic.

Network dispatching was implemented using `system("curl...")` calls to execute headless HTTP POST requests, successfully mimicking a Wi-Fi enabled ESP8266 or Arduino IoT microcontroller.

6. Integration and Testing

6.1 Description of the Integration Modules

The AquaSense system architecture relies on the seamless integration of three primary, decoupled modules. The integration strategy utilizes standard HTTP protocols and local file I/O to ensure modularity and fault tolerance:

1. Hardware-to-Backend Integration (Sensor Ingestion)

This module integrates the C++ Hardware Simulator with the Python Flask Server. The integration is achieved over the local network via stateless HTTP POST requests. The C++ simulator acts as the client, serializing randomized flow-meter data into a standard JSON payload and pushing it to the /api/data endpoint at a frequency of 0.5Hz. The backend parses this JSON, integrating the raw hardware telemetry into the core Python logic engine.

2. Backend-to-Storage Integration (Data Persistence)

To ensure historical data is preserved for analytics without the overhead of a complex SQL database, the Python backend integrates directly with the host machine's file system. During each data cycle, the aggregation engine flushes the current state of the global Python dictionary into history.csv. This integration handles thread-safe appending of comma-separated values to ensure data integrity over long-term operation.

3. Backend-to-Frontend Integration (Dashboard Analytics)

This module connects the user-facing HTML/JavaScript interface to the Flask REST API. The integration is handled purely via asynchronous JavaScript fetch() API calls. The dashboard makes periodic HTTP GET requests to the backend endpoints (/api/data for live stats, and /api/reports for historical charts). By decoupling the frontend from the backend, the integration ensures the web UI remains highly responsive, dynamically updating DOM elements and Chart.js canvases without requiring a full page reload.

6.2 Testing Methodology

To verify the robustness and accuracy of the system, various levels of testing were performed. The testing phase focused on validating normal telemetry ingestion, edge-case anomaly detection (leaks), and remote command responsiveness.

6.2.1 Unit Testing

Unit testing focused on validating individual components and logic functions in isolation to ensure they produce correct outputs for given inputs.

Test ID	Component	Test Scenario	Input Data	Expected Outcome	Pass/Fail
UT-01	Python Backend	Aeration Scaling Logic	`tank_level = 10%`	Aeration set to 90%	Pass
UT-02	Python Backend	CSV Logging Function	`system_data` object	Row appended to `history.csv`	Pass
UT-03	C++ Simulator	Random Flow Generation	Base flow = 5.0	Output within 10% variance	Pass
UT-04	JS Frontend	Unit Conversion	Value: 1000ml	Display: "1.0 L"	Pass
UT-05	Python Backend	Alert Trigger Logic	Flow > 12.0 L/m	"High Flow" alert string generated	Pass

6.2.2 Integration Testing

Integration testing verified the communication between different modules, ensuring data integrity is maintained across system boundaries.

Test ID	Interface	Test Scenario	Input Data	Expected Outcome	Pass/Fail
IT-01	C++ -> Flask	Sensor Data Ingestion	JSON: `{"main_flow": 10.0}`	HTTP 200 OK; Backend data updated	Pass
IT-02	Flask -> CSV	Persistence Validation	Data Update Event	File `history.csv` size increases	Pass
IT-03	Flask -> JS	API Data Retrieval	GET `/api/data`	Valid JSON received by frontend	Pass
IT-04	Flask -> Telegram	Alert Notification	Critical Leak Trigger	Push notification received on mobile	Pass
IT-05	Telegram -> Flask	Remote Command	`/area control kitchen valve off`	Kitchen valve status set to False	Pass

6.2.3 System Testing (End-to-End)

System testing validated the complete workflow from sensor detection to user notification and automated control.

Test ID	Scenario Description	Input/Trigger	Expected System Response	Actual Outcome	Pass/Fail
ST-01	Normal Operation	Valid Sensor Data	Dashboard updates every 2s; Charts reflect live usage.	As expected	Pass
ST-02	Critical Tank Level	Level < 20%	Auto-Mode triggers; Flow restricted; Critical alert sent.	As expected	Pass
ST-03	Leak Detection	Flow > 14 L/m	"High Flow Anomaly" alert; Valve automatically capped at 20%.	As expected	Pass
ST-04	Supply Cut Fallback	`main_flow = 0.0`	Dashboard shows "Water Cut"; System enters conservation.	As expected	Pass
ST-05	Error Resilience	Empty POST Request	HTTP 400 Error; Server remains stable and active.	As expected	Pass
ST-06	Telegram Monitoring	`/report` command	Bot replies with Tank Level, Usage, and Auto-Mode status.	As expected	Pass

6.3 Conclusion

The testing phase confirmed that the AquaSense system is highly resilient. Unit tests verified the mathematical accuracy of the conservation logic, integration tests confirmed seamless communication between C++ and Python modules, and system tests proved that the end-to-end automation—including Telegram notifications and Auto-Mode throttling—functions correctly under various environmental conditions.

7. Conclusion

The **AquaSense Smart Water Management System** successfully demonstrates a highly scalable, real-time solution to the growing global challenge of water scarcity and domestic water wastage. By bridging the gap between localized hardware telemetry and modern web analytics, this project proves that intelligent monitoring can drastically reduce resource consumption without severely impacting user experience.

Through the implementation of a decoupled architecture, the system achieved all of its primary objectives:

1. **Real-time Monitoring:** The C++ simulated edge nodes successfully ingested and dispatched high-frequency flow-rate telemetry to the backend without latency issues.
2. **Automated Conservation:** The Python-based logic engine accurately detected critical tank thresholds and municipal supply cuts, dynamically scaling aeration percentages to force water conservation automatically.
3. **Responsive User Experience:** The integration of an asynchronous, glassmorphism-styled web dashboard allowed users to view complex time-series data and trigger manual valve overrides seamlessly.

Future Scope:

While the current mini-project relies on a C++ software simulator to mimic hardware interrupts, the modular design ensures that physical IoT microcontrollers (such as an ESP32 or Raspberry Pi connected to physical solenoid valves and YF-S201 flow sensors) can be integrated in the future with zero changes required to the backend API. Furthermore, future iterations of AquaSense could migrate the local CSV logging to a cloud-hosted SQL database, and implement Machine Learning (ML) algorithms to predict a household's weekly water usage patterns based on historical data.

Overall, AquaSense provides a robust, foundational blueprint for building modern, eco-friendly smart home architectures.

8. References

1. Grinberg, M. (2018). Flask Web Development: Developing Web Applications with Python (2nd ed.). O'Reilly Media. (Used for standardizing the REST API architecture and Python routing logic).

2. Mozilla Developer Network (MDN). (2023).Fetch API Documentation & Asynchronous JavaScript. Retrieved from https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API. (Used as the primary reference for building the non-blocking frontend dashboard).

3. Chart.js Contributors. (2023). Chart.js Documentation: Simple HTML5 Charts. Retrieved from https://www.chartjs.org/docs/latest/. (Used for implementing the responsive time-series canvas visualizations).

4. Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications. IEEE Communications Surveys & Tutorials, 17(4), 2347-2376. (Referenced for theoretical edge-node decoupling and HTTP payload transmission standards).

5. Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley Professional. (Referenced for utilizing standard `<chrono>`, `<thread>`, and `<random>` libraries in the hardware simulation engine).